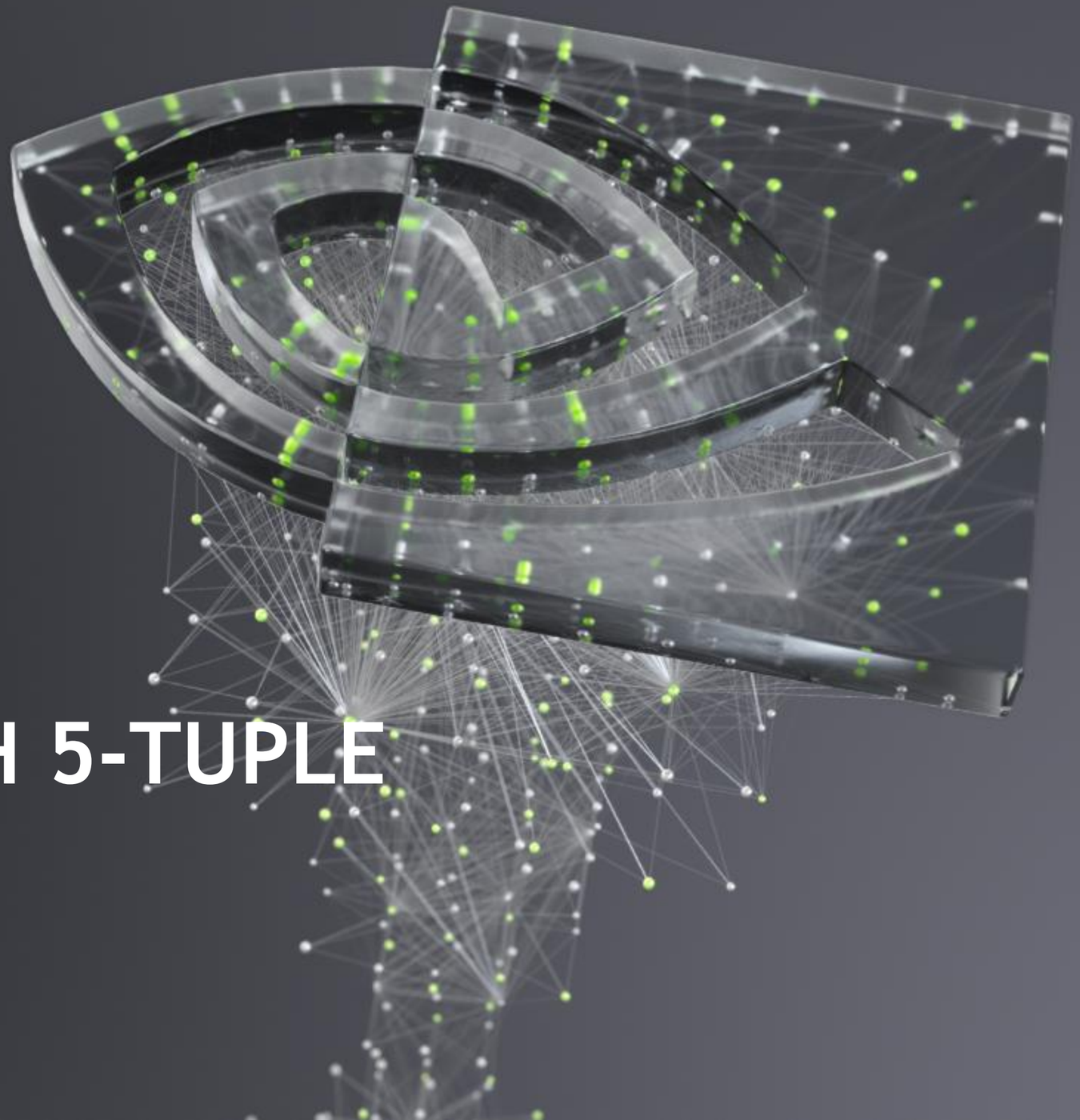


EXTENDING TC WITH 5-TUPLE HASH OFFLOAD

Roni Bar Yanai, Rony Efraim, Jul 2020

AGENDA

- Programmable data plane: TC and TC HW offload recap
- Hash use cases: ECMP, Service Load Balancing
- TC 5-tuple hash extension

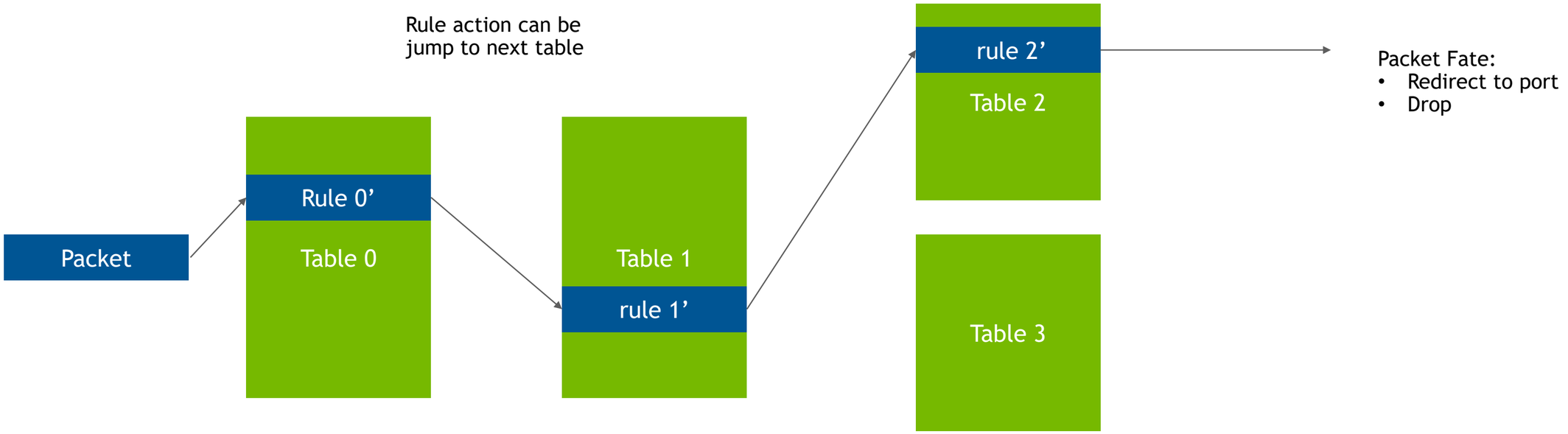
NVIDIA.



TC & TC OFFLOAD RECAP

PROGRAMABLE DATAPATH

- Consisted of tables
- Each table has rules that are built from matches and actions
- Rules and table can be created and destroyed dynamically according to SDN controller, and traffic seen



PROGRAMABLE DATAPATH

- TC Flower classifier has a flexible key
- TC action, can modify and steer packet
- With the above, TC is expressive enough, flexible enough and provide a programable data plane. TC API is vendor agnostic and is used for HW offload of programmable data path.

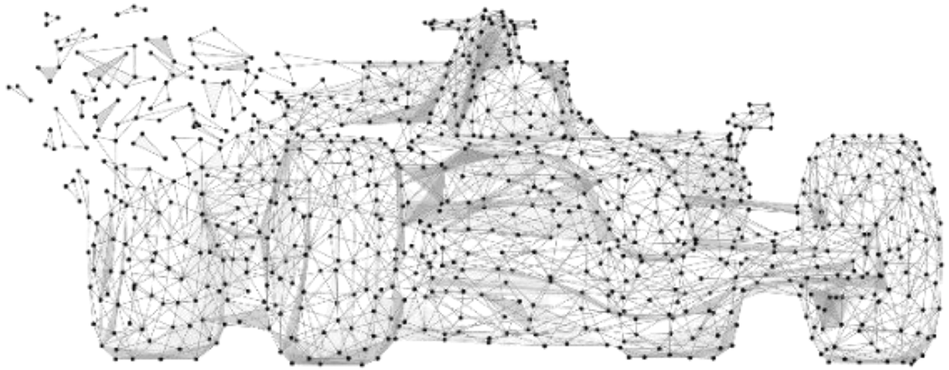
```
#tc filter add dev vxlan0 protocol ip parent ffff: piro 1 \
        flower \
        dst_mac e4:11:22:33:45:61 \
        src_mac e4:11:22:33:46:61 \
        enc_src_ip 1.1.1.1 \
        enc_dst_ip 2.2.2.2 \
        enc_dst_prt 4789 \
        enc_key_id 16 \
        action tunnel_key unset \
        action mirred egress redirect dev ens1f0_0
```


NVIDIA.

PROGRAMABLE DATAPATH OFFLOAD

Test	Offload
1 Flow VXLAN + CT	46MPPS-70 MPPS
200k Flows VXLAN + CT	27 MPPS

- Packet size 114 (VXLAN + 64 bytes packet inner)
- Packets are sent in round robin
- ConnextX-6DX
- Zero CPU cycles

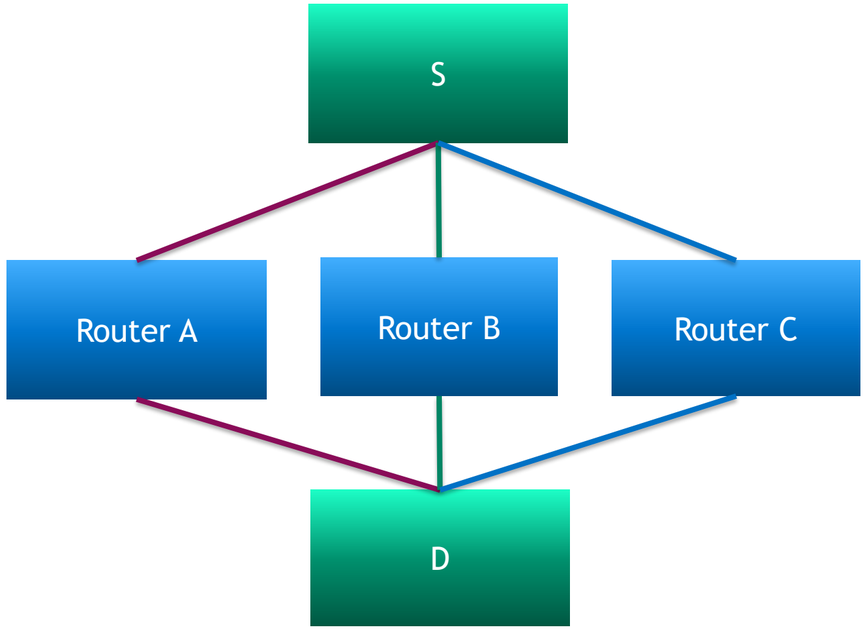




ECMP – EQUAL COST MULTI PATH

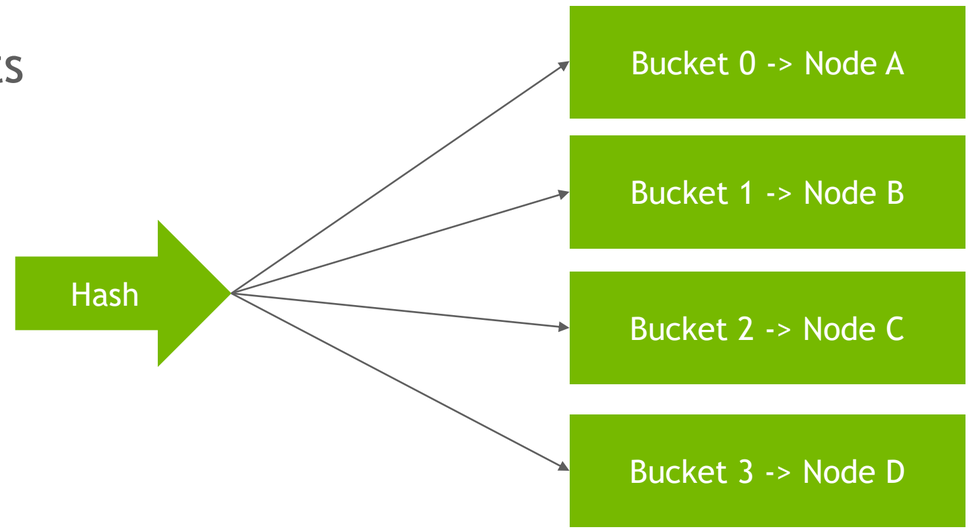
ECMP – EQUAL COST MULTI PATH

- Packet has multiple best paths
- Balancing between paths will improve network utilization
- Each node might have different load, hence different latency and packet might arrive out of order
- ECMP must enforce per session consistency



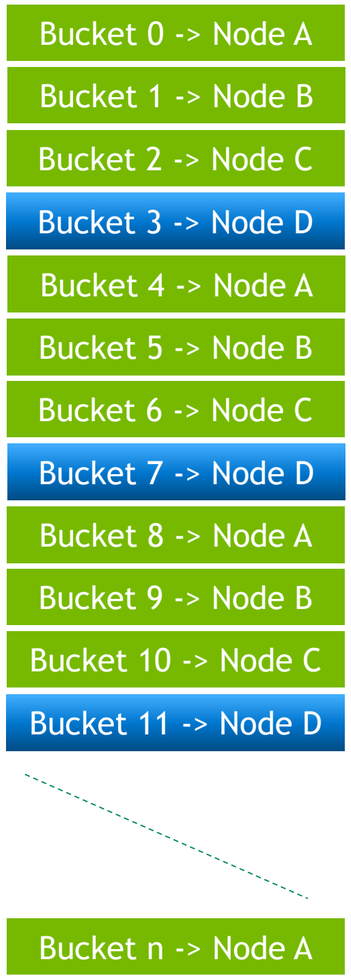
ECMP – EQUAL COST MULTI PATH

- Allocating and maintaining sessions per 5-tuple has performance and memory impact
- In case of update all entries should be scanned and updated
- A much more efficient approach is hashing and executing per bucket
- For each packet we calculate hash on 5-tuple
- We continue the execution on one of the buckets according to hash/mask
- Buckets can updated separately



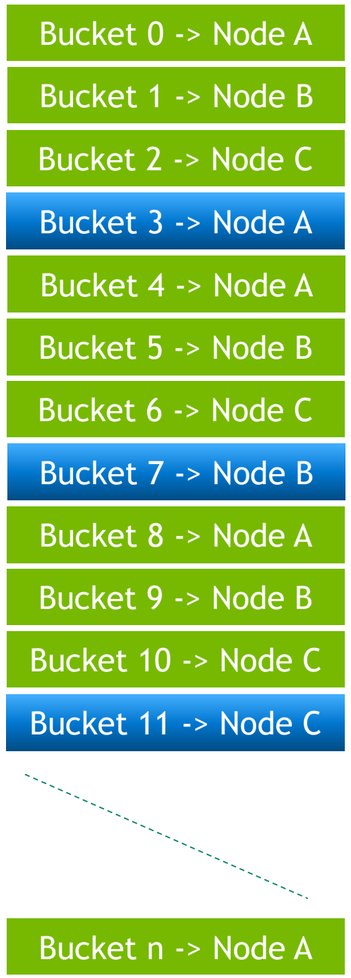
ECMP – EQUAL COST MULTI PATH

- Consistent hashing - on update k/n of the sessions are moved (k is number of buckets, n is number of nodes)
- Higher number of buckets can give better distribution and also support weights
- When node D is removed, update k/n of the nodes
- The updated buckets will cause a one time out of order for already existing sessions



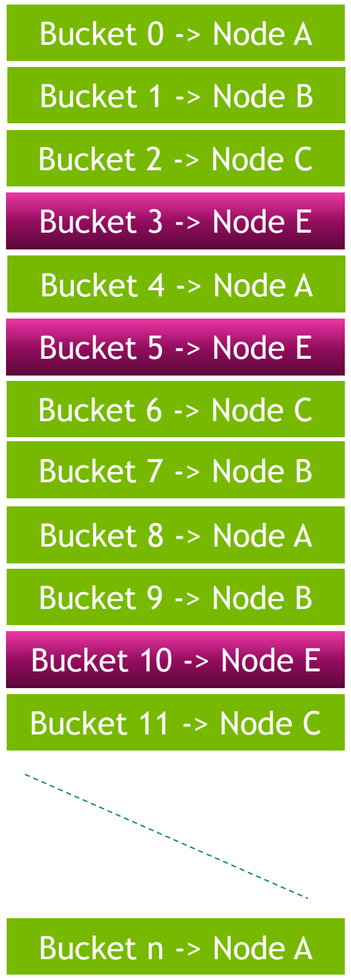
ECMP – EQUAL COST MULTI PATH

- Node D was removed
- Every bucket with Node D was replaced by one of the other existing Nodes in a balanced way
- Traffic is balanced
- Only sessions that were destinated to bucket D were moved
- Adding Node will spilt traffic from all existing Nodes



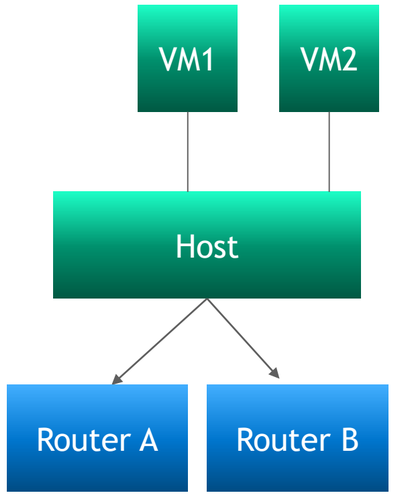
ECMP – EQUAL COST MULTI PATH

- Node E added, update k/n
- We chose X buckets and update them
- Traffic is balanced
- Assumption that adding/removing nodes rate is slow in factor to sessions open/close rate and duration



ECMP – GENERALIZATION

- ECMP is just one use case of load balancing
- Using hashing and buckets can be use for other use cases:
  - Redundancy Active-Active
  - Service load balancing
  - Gateway



VM is unaware of the redundancy

Host has Active-Active Redundancy



EXTENDING TC WITH HASH

EXTENDING TC WITH HASH

- Add TC hash action that hash on 5-tuple
- Add flower hash match/mask

```
$ tc filter add dev ens1f0_0 ingress \
   prio 1 chain 0 proto ip \
  flower ip_proto tcp \
  action hash \
  action goto chain 2

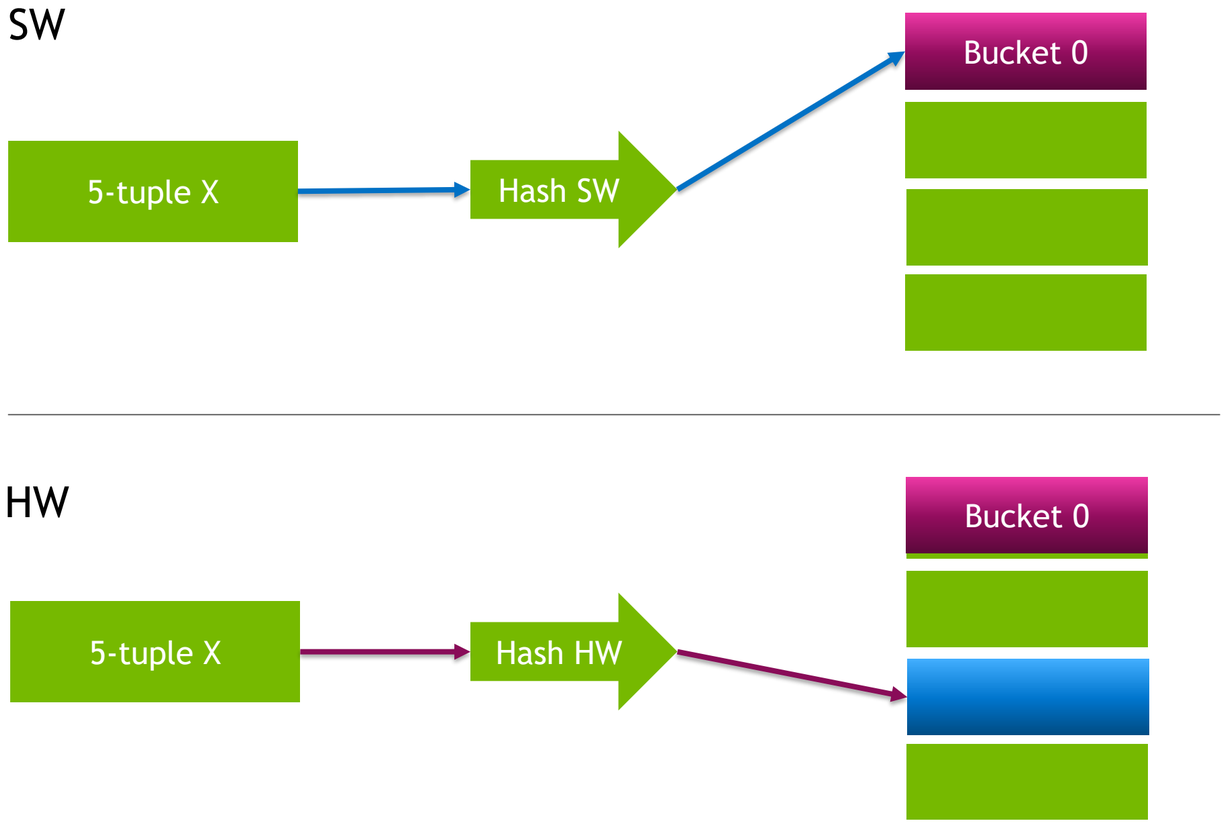
$ tc filter add dev ens1f0_0 ingress \
prio 1 chain 2 proto ip \
flower hash 0x1/0xf  \
action mirred egress redirect dev ens1f0_1
```

skb->hash = hash_func(skb)

NVIDIA.

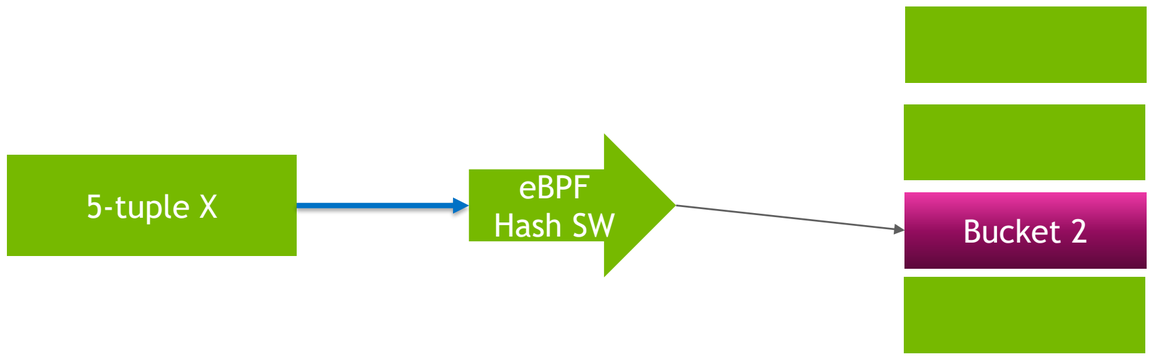
EXTENDING TC WITH HASH HW OFFLOAD

- HW hash might be different then SW hash
  - Application might use its own hash
  - HW might chose hash that is more efficient
  - SW might change the behavior
- In use cases where HW rules are generated dynamically by SW according to traffic seen, hash might have different implementation and therefor will select a different bucket
- HW rule will direct to bucket 2, that is empty, and packet will be directed again to SW. This will not converge in some cases
- If rules are inserted statically and all traffic always run in HW, there is no consistency problem



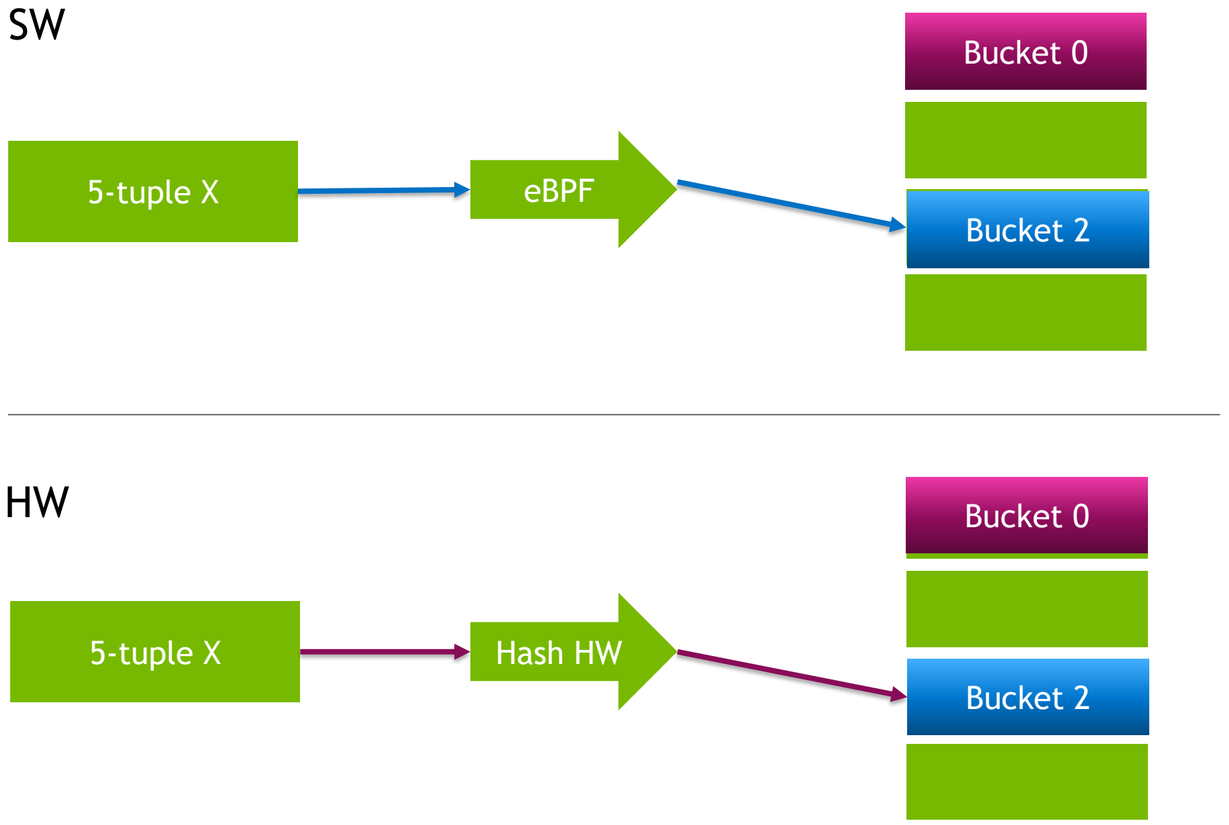
EXTENDING TC WITH HASH: USE EBPF

- Syncing the SW hash and HW hash can be done by BPF
- When using hash action we also provide the hash function in BPF format
- Every vendor can generate its own hash version
- Hash BPF can be located under /sys/class/net/…



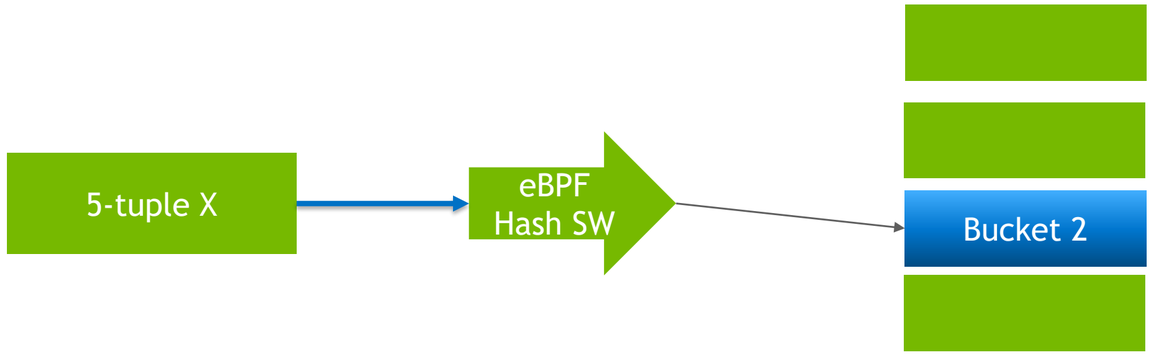
EXTENDING TC WITH HASH HW OFFLOAD

- HW and SW are in sync



EXTENDING TC WITH HASH

```
$ tc filter add dev ens1f0_0 ingress \
  prio 1 chain 0 proto ip \
  flower ip_proto tcp \
  action hash bpf object-file <file> \  action goto chain 2
```



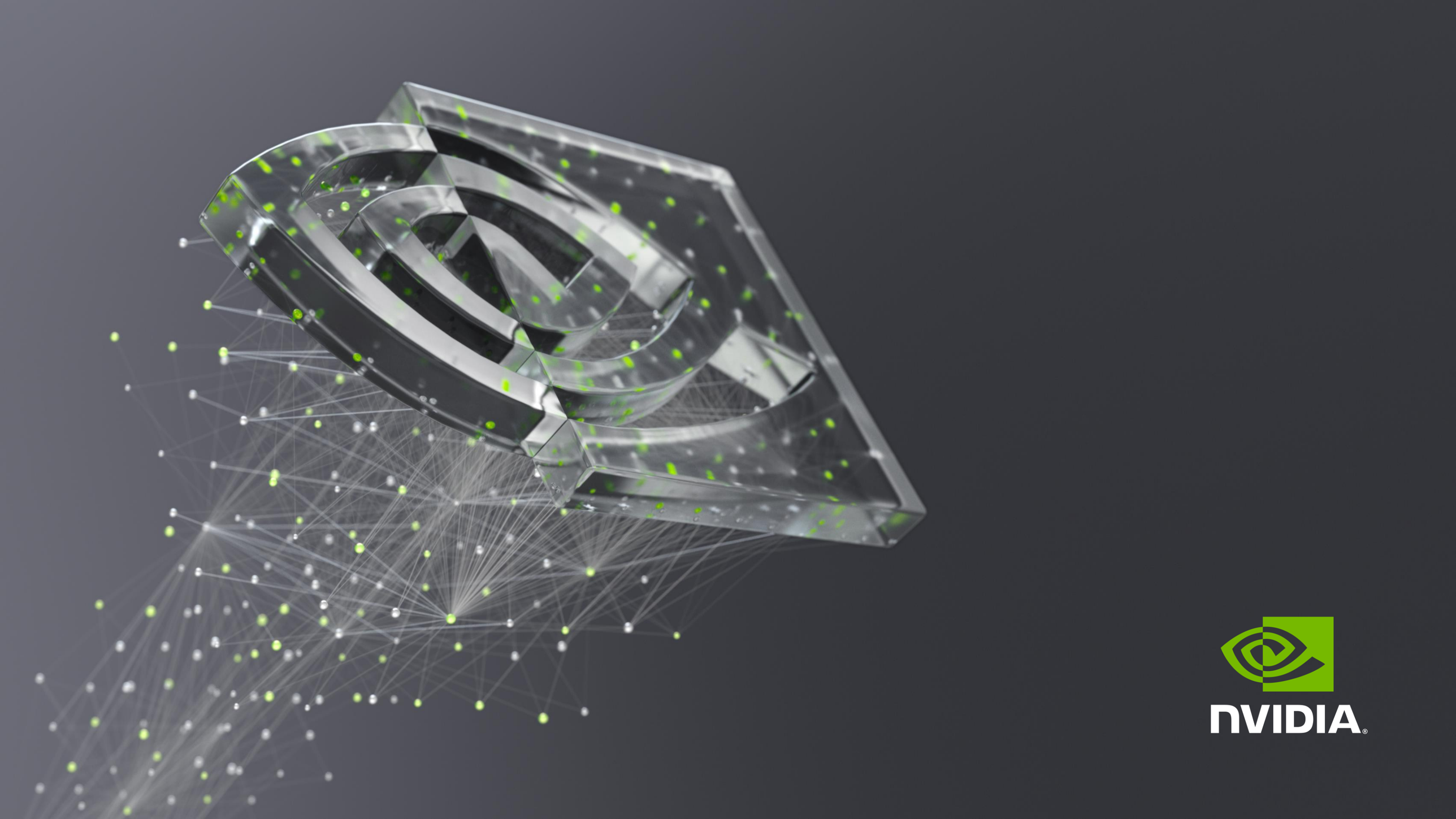
SUMMARY

- Programmable data plane using TC and TC HW offload
- Hash use cases
- Enabling TC and TC HW offload for hash use cases, by extending TC expressiveness
- HW and SW sync challenge
- Executing hash by eBPF can solve the sync problem
- Patches:

  https://www.spinics.net/lists/netdev/msg667690.html

- Discussion: bpf action already exists, why not use that?
  - bpf can implement many of TC actions
  - give user the expressiveness without being too generic is the tradeoff

NVIDIA.


NVIDIA